Pattern Matching Basics

Elixir doesn't have a normal assignment operator. In most languages, the = operator is used for simple left = "value" statements. In Elixir, this operator is called the match operator, and it initiates pattern matching.

Pattern matching is implemented by the BEAM, so it's baked into the runtime of the language. Some optimizations make it efficient even for a large number of pattern clauses. So, you can use pattern matching without worrying about a negative performance impact on your application.

In this section, we'll go over different pattern-matching syntaxes for basic data types, lists, maps, tuples, and more.

Match Basic Types

Let's start with the most basic syntax for pattern matching. Open a new IEx session and type the following:

```
$ iex
iex> 1 = 1
1

iex> a_number = 1
1

iex> 1 = a_number
1
```

This first example is seemingly simple, but 1 = 1 is rather unusual. In Ruby, you can only have variables on the left side of =. Clearly, that's not the case here.

To evaluate a pattern mentally, execute the right side and then compare the result with the left side. Assign any variables that are on the left side. If the patterns don't match, then you get a MatchError:

```
iex> 1 = 2
** (MatchError) no match of right hand side value: 2
```

Variable assignment works just as it does in Ruby. Values are reassigned when they're on the left side:

```
iex> a_number = 1
iex> a_number = 2
iex> a_number
2
```